

FitFlow

Une application de coaching sportif,
du suivi en salle à l'analyse de la progression

Marc Yrius — Titre professionnel Concepteur Développeur d'Applications
RNCP 37873 · Niveau 6 · Session 2026

Déroulé

1 Contexte

Le problème · Les utilisateurs · L'objectif · La problématique

2 Besoin et conception

Les personas · Les besoins · Les maquettes · L'architecture

3 Réalisation

Backend · Les trois clients · Démonstration — parcours type · Ce que la démo a montré

4 Conduite de projet

Les jalons · Depuis le dossier · La méthode

5 Base de données

PostgreSQL · Le modèle · Accès aux données

6 Sécurité

Mots de passe · Jetons JWT · Rôles · Validation

7 Tests

Volume et niveaux · Intégration · Cas de sécurité

8 Déploiement et DevOps

Environnement · Intégration continue · Mise en production

9 Bilan

Difficultés · Solutions · Limites · Évolutions

PARTIE 1

Contexte

1. Le problème
2. Les utilisateurs
3. L'objectif
4. La problématique

Un coach jongle avec quatre outils pour suivre un seul client

Excel

Les séances, les charges, la progression

Son téléphone personnel

Les échanges, les rappels, les photos

Les réseaux sociaux

Certains clients ne passent que par là

Google Agenda

Les rendez-vous, séparés de tout le reste

Deux utilisateurs, deux usages opposés

Le coach

Prépare, distribue, suit, analyse. Il travaille **entre les séances**, souvent le soir, sur un ordinateur.

Le client

Exécute et saisit. Il travaille **pendant la séance**, debout, sur un téléphone, entre deux séries.

Le même produit, deux contextes d'usage qui n'ont **rien en commun**.

L'objectif

*Le coach n'est pas un professionnel de la tech.
C'est à l'outil de se simplifier, pas à lui de s'adapter.*

Centraliser dans une seule application ce qui est aujourd'hui éparpillé — et le rendre **assez simple pour qu'il n'y ait rien à apprendre.**

La problématique

Comment concevoir une application unique qui réponde aux besoins de deux utilisateurs aux usages opposés, sans qu'aucun des deux n'ait à s'adapter à l'outil ?

PARTIE 2

Besoin et conception

1. Les personas
2. Les besoins
3. Les maquettes
4. L'architecture

Deux personas guident chaque arbitrage

Thomas — coach en salle, à son compte

Une douzaine de clients. Passe plus de temps à recopier qu'à programmer.

Ce qu'il veut : ne plus ressaisir la même séance dix fois.

Camille — cliente débutante

Peu de repères en musculation. Veut progresser sans se blesser, et savoir si ce qu'elle fait sert à quelque chose.

Ce qu'elle veut : qu'on lui dise quoi faire, et voir que ça avance.

Les besoins, traduits en exigences

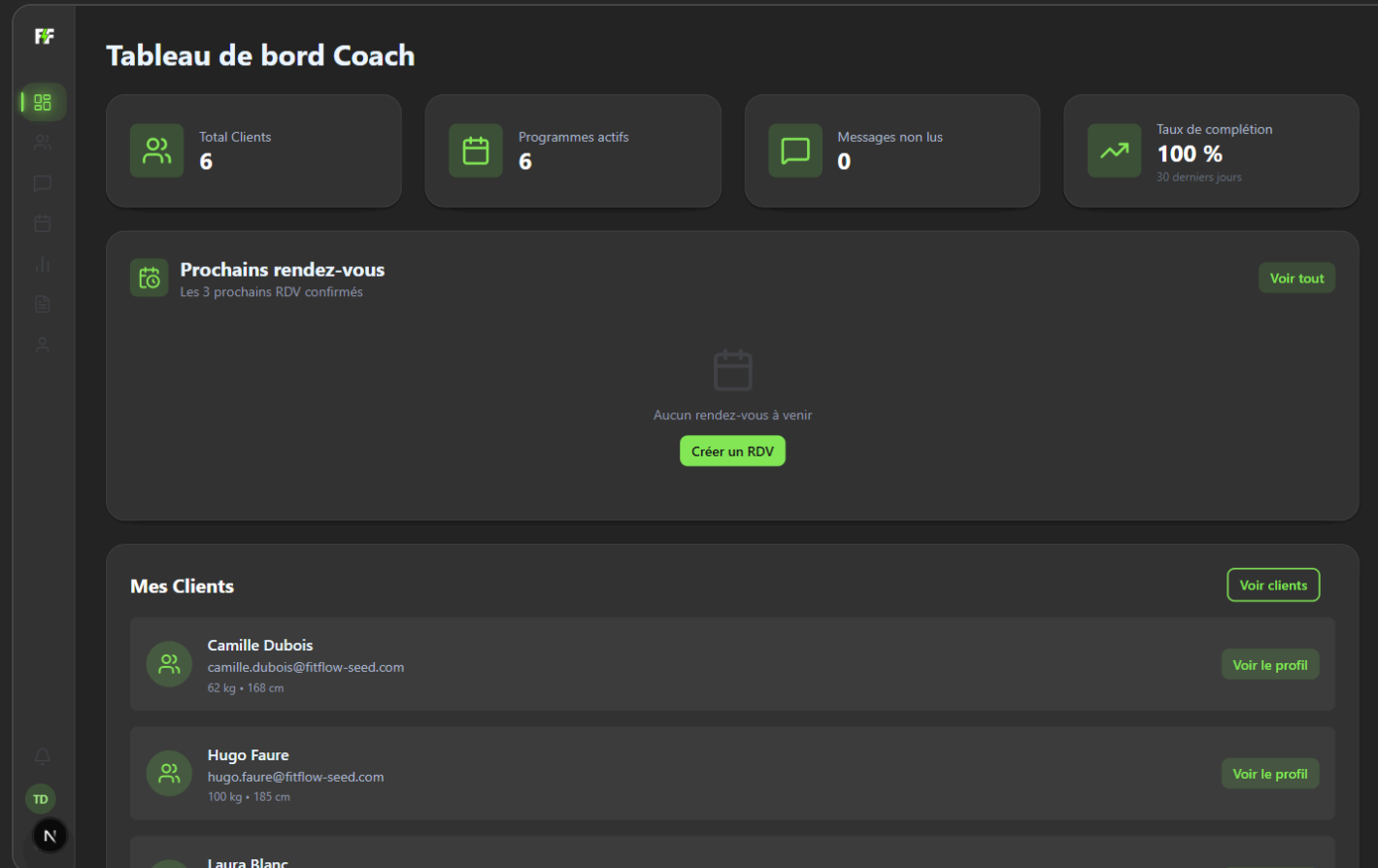
Côté coach

- Créer et réutiliser des séances
- Suivre la progression chiffrée
- Échanger via FitFlow
- Récupérer ses données

Côté client

- Saisir une série en trois gestes
- Trouver un coach par salle ou par ville
- Suivre son poids, son sommeil, ses repas
- Être rassuré : les coachs sont notés

Des maquettes au produit — et pourquoi elles n'ont pas suivi



Les écrans clés ont été maquetés avant d'être codés. Le produit s'en est éloigné au fil des itérations, et je n'ai pas maintenu les maquettes à jour : **deux référentiels qui divergent valent moins qu'un seul qui est vrai**. Ce que vous voyez est l'application réelle, en production.

Une architecture trois tiers

Clients

Dashboard web
Application mobile

API REST

Node.js / Express
Seule source de vérité

Données

PostgreSQL
Stockage objet

Aucun client n'accède directement à la base. Toute règle métier vit dans l'API — c'est ce qui garantit que le web et le mobile se comportent pareil.

PARTIE 3

Réalisation

1. Backend
2. Les trois clients
3. Démonstration — parcours type
4. Ce que la démo a montré

Backend — Node.js, Express, Prisma

Structure

25 modules de routes, 133 points d'entrée, 27 contrôleurs. Une route déclare son rôle requis, le contrôleur vérifie la propriété.

Prisma comme ORM

Requêtes paramétrées, migrations versionnées, typage généré depuis le schéma.

Réponses uniformes

```
{ success: true,  
  message: "...",  
  data: { ... } }
```

Les deux clients traitent toutes les réponses de la même manière, quel que soit l'endpoint.

Trois clients, une seule source de vérité

Dashboard web

Next.js 16, React 19, TypeScript.
Le poste de travail du coach, et un accès client allégé.

Application mobile

React Native, Expo.
Le client en salle, et le coach sur place : onze écrans lui sont dédiés.

API

Toute règle métier vit ici.
Les deux clients se comportent forcément pareil.

Le jeton d'authentification est conservé dans le **trousseau système** sur mobile, pas dans un stockage ordinaire.

Parcours utilisateur type

Du programme préparé par le coach à la donnée qui lui revient.

Ce que ce parcours démontre

La boucle est fermée

Ce que le coach prépare, la cliente l'exécute, et la donnée lui revient **sans ressaisie**.

La saisie est pensée pour la salle

Trois gestes par série. Le renforcement se saisit en **temps tenu**, pas en répétitions — un gainage de 45 secondes compté comme 45 répétitions fausserait tout le volume.

Le template capitalise

Répondre vite au cas courant, **sans empêcher le cas particulier** : une séance reste créable de toutes pièces.

Et l'analyse suit

Volume, 1RM estimé, INOL, cartographie — plus un export tableur, parce que je ne force personne à quitter Excel.

PARTIE 4

Conduite de projet

1. Les jalons
2. Depuis le dossier
3. La méthode

Huit mois, quatre phases

Janvier — février

Le socle : API, modèle de données, dashboard, application mobile. Premier commit le 13 janvier.

Mars — avril

Le métier : séances, analyses de musculation, rendez-vous, onboarding. Premiers tests d'intégration et première chaîne d'intégration continue.

Mai — juillet

Creux assumé : trois commits en trois mois. Le projet n'a pas avancé.

Août — septembre

Durcissement puis exploitation : sécurité, conteneurisation, modération, mise en production en août. **110 commits** au total.

Ce qui a changé depuis le dossier

Modération complète

Signalement, sanctions graduées, recours, suppression différée. Conçue à partir de la contrainte RGPD, pas ajoutée après.

Un écran d'analyse qui ne trouvait aucun client

Une requête restée sur le champ hérité d'avant le multi-coach. Corrigée, et couverte par quatre tests de non-régression.

225 exercices reclassés en production

Gainage et chaise comptés en répétitions faussaient le volume. Requalifiés en renforcement, sans perdre une séance validée.

Messagerie en temps réel

WebSocket en accélérateur, le sondage conservé en file. Déployé cette semaine.

La méthode, et ce qu'elle a coûté

Ce qui a tenu

Une fonctionnalité à la fois, menée jusqu'au bout : API, dashboard, mobile. Les décisions structurantes écrites dans le dépôt, à côté du code.

Trois dépôts

Choix retenu sur les conseils de mon formateur. Chacun a sa chaîne d'intégration, mais le report manuel entre les trois m'a coûté cher.

Ce qui a manqué

Pas de suivi formel.

Ce que je referais

Un dépôt unique, et des jalons écrits.

PARTIE 5

Base de données

1. PostgreSQL
2. Le modèle
3. Accès aux données

PostgreSQL, hébergé sur Supabase

Pourquoi relationnel

Les données sont fortement liées : un client, ses coaches, ses programmes, ses séances, ses séries. Les contraintes d'intégrité font le travail à ma place.

Pourquoi PostgreSQL

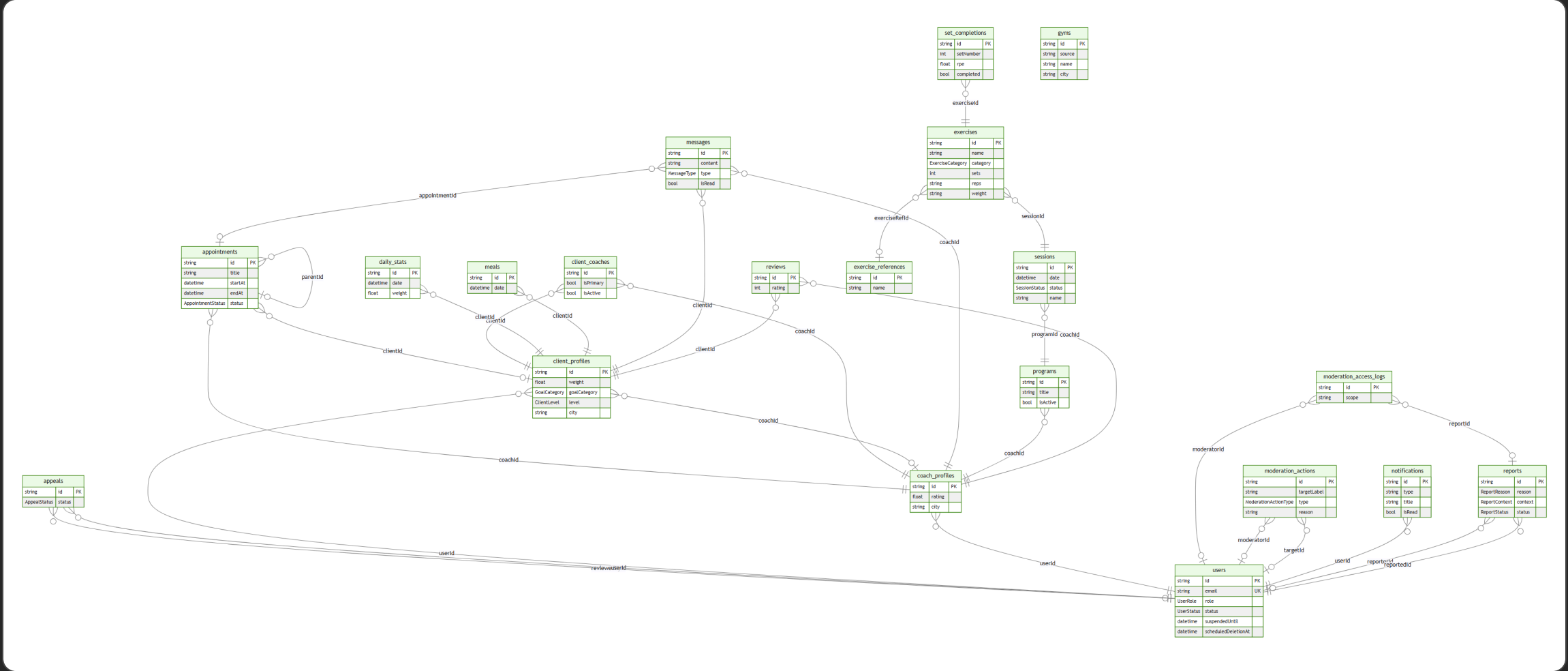
Types énumérés natifs, contraintes composites, et un service managé gratuit.

Une exception assumée

Un **cache Redis** pour le compteur de notifications, appelé toutes les 30 secondes par chaque application ouverte.

Lecture traversante, invalidation à l'écriture, et **dégradation propre** : si Redis tombe, l'application ralentit, elle ne s'arrête pas.

Le modèle — 36 entités



L'accès aux données

Requêtes paramétrées

Prisma ne concatène jamais de chaîne SQL : les valeurs passent en paramètres. **L'injection SQL est structurellement impossible.**

Projections explicites

Chaque requête déclare les champs qu'elle retourne. Un profil public ne peut pas laisser fuir un email par inadvertance : il n'est pas dans la projection.

37 relations en cascade : supprimer un compte emporte ses données liées, sans script de nettoyage à maintenir.

PARTIE 6

Sécurité

1. Mots de passe
2. Jetons JWT
3. Rôles
4. Validation

Les mots de passe ne sont jamais stockés

bcrypt

Empreinte à sel intégré, coût paramétrable. Même mot de passe, empreintes différentes pour deux comptes.

Volontairement lent : c'est ce qui rend une attaque par force brute coûteuse.

Ce qui en découle

Une fuite de la base ne livre pas les mots de passe.

L'empreinte est **exclue de l'export RGPD** : ce n'est pas une donnée fournie par la personne, et l'exporter reviendrait à distribuer une cible.

Les jetons JWT

Ce qu'ils portent

L'identifiant et le rôle, signés. Le serveur ne garde pas de session : il vérifie la signature à chaque appel.

Où ils sont rangés

Trousseau système sur mobile. `localStorage` sur le web — **une faiblesse que j'assume.**

La limite, dite avant qu'on me la demande

Jeton valable 30 jours, **non révocable** : la déconnexion ne l'invalide pas côté serveur.

Le correctif est identifié — jeton court plus jeton de rafraîchissement, ou liste de révocation. Il n'est pas implémenté.

L'autorisation se joue à deux niveaux

1 — Par rôle, sur la route

`authorize('COACH')` écarte immédiatement un client qui appellerait une route de coach.

2 — Par propriété, dans le contrôleur

```
if (program.coachId !== coachProfile.id)
  return sendError(res, 'Accès non autorisé',
    403);
```

Le rôle ne suffit pas. Deux coachs ont le même rôle mais pas les mêmes clients : sans le second niveau, n'importe quel coach lirait le programme d'un confrère en devinant un identifiant.

Validation et défenses de surface

Validation des entrées

Écrite à la main dans les contrôleurs — **aucune bibliothèque**. La couverture est donc inégale, et je le sais.

Le correctif

Un schéma de validation par route, et un 400 systématique plutôt qu'une erreur 500.

Au niveau du serveur

- **helmet** — en-têtes de sécurité sur toutes les réponses
- **Limitation de débit** — 10 tentatives ratées par IP et par quart d'heure sur la connexion
- Seuls les *échecs* sont comptés : un usage normal n'est jamais gêné

PARTIE 7

Tests

1. Volume et niveaux
2. Intégration
3. Cas de sécurité

220 tests, sur quatre niveaux

132

intégration, sur base réelle

42

unitaires

41

composants d'interface

5

bout en bout, navigateur

Le backend utilise le runner natif node : test, le dashboard Vitest, et Playwright pour le navigateur.

Les tests d'intégration tournent sur une vraie base

Comment

Une base PostgreSQL dédiée, recréeée vierge, un serveur en `NODE_ENV=test` sur un port séparé. Les tests appellent l'API **en HTTP**, comme un client.

Pourquoi pas un simulacre

Un faux Prisma ne teste que mes hypothèses sur Prisma.

Une contrainte d'unicité oubliée ne se voit que sur une vraie base.

Le cloisonnement est testé, pas supposé

La suite construit **deux écosystèmes complets et indépendants** — un coach A avec ses clients, un coach B avec les siens — puis vérifie que B n'atteint **aucune** ressource de A.

Ce qui est vérifié

- Lire un programme d'autrui → **403**
- Lire une conversation d'autrui → **403**
- Un client appelant une route coach → **403**

La forme

Une table de cas plutôt que des tests écrits un par un : protéger un nouvel endpoint revient à ajouter une ligne.

PARTIE 8

Déploiement et DevOps

1. Environnement
2. Intégration continue
3. Mise en production

L'environnement de développement

Docker Compose

PostgreSQL et Redis en conteneurs. Le même socle pour tout le monde, et **rien à installer sur la machine**.

Trois dépôts Git

Backend, dashboard, mobile. Un changement dans l'un ne casse pas les autres — choix retenu sur les conseils de mon formateur.

L'intégration continue

Deux chaînes GitHub Actions

- Backend : PostgreSQL conteneurisé, migrations, puis la suite complète
- Dashboard : lint, tests, build, puis les tests bout en bout

Ce que j'ai découvert en les regardant

Elles étaient **rouges sur la totalité de leurs exécutions**, depuis des mois.

Une intégration continue qui échoue toujours finit par être ignorée : **elle ne protège plus rien.**

Trois causes racines diagnostiquées et corrigées. **Les deux chaînes sont vertes** — ce sont les premières exécutions réussies de l'historique.

La mise en production

Vercel

Dashboard

déploiement à chaque push

Fly.io

API

deux machines, région Paris

Supabase

PostgreSQL et stockage

région Irlande

L'ordre compte : les migrations d'abord, le code ensuite. Déployer du code qui attend une colonne absente casse toutes les connexions.

L'application est en ligne, pour 0 € par mois.

PARTIE 9

Bilan

1. Difficultés
2. Solutions
3. Limites
4. Évolutions

La difficulté principale — une erreur de modélisation

AU DÉPART

Un client → **un** coach

Simple à modéliser. Une clé étrangère, rien de plus.



APRÈS

Un client → **plusieurs** coaches

Table de liaison, coach principal, spécialités distinctes.

Un client peut vouloir un coach pour la musculation **et** un autre pour la nutrition. **Le modèle interdisait un besoin réel** — il a fallu le reprendre tard, et en assumer le coût.

Les solutions, et ce qu'elles m'ont coûté

Le report manuel entre dépôts

Je modifiais le dashboard et le backend ensemble, puis j'oubliais le mobile. Le format d'envoi changeait, et le mobile cassait.

La parade était disciplinaire, pas technique : reporter sur le mobile immédiatement, avant de passer à autre chose.

Vérifier plutôt que supposer

Sur une lenteur d'API, j'avais une explication cohérente — la distance entre l'API et la base. **Elle était fausse.**

Trente secondes de mesure l'ont réfutée : le coût venait du mode de connexion. **Une cause plausible n'est pas une cause démontrée.**

Les limites, telles qu'elles sont

- **La qualité des données dépend de l'humain.** Si le client saisit mal, l'analyse est fausse.
- **Aucun test utilisateur réel.** Des contacts avec des coachs, mais pas de phase de test — et sur iPhone la distribution était bloquée sans licence développeur.
- **L'imbrication avec les outils existants reste partielle**, faute d'accès à tout ce que les coachs utilisent.

Ce qui suivrait

La facturation

Au centre de l'idée de départ, pas encore en place.

Analyse des repas par photo

Écartée : trop coûteuse pour la valeur rendue.

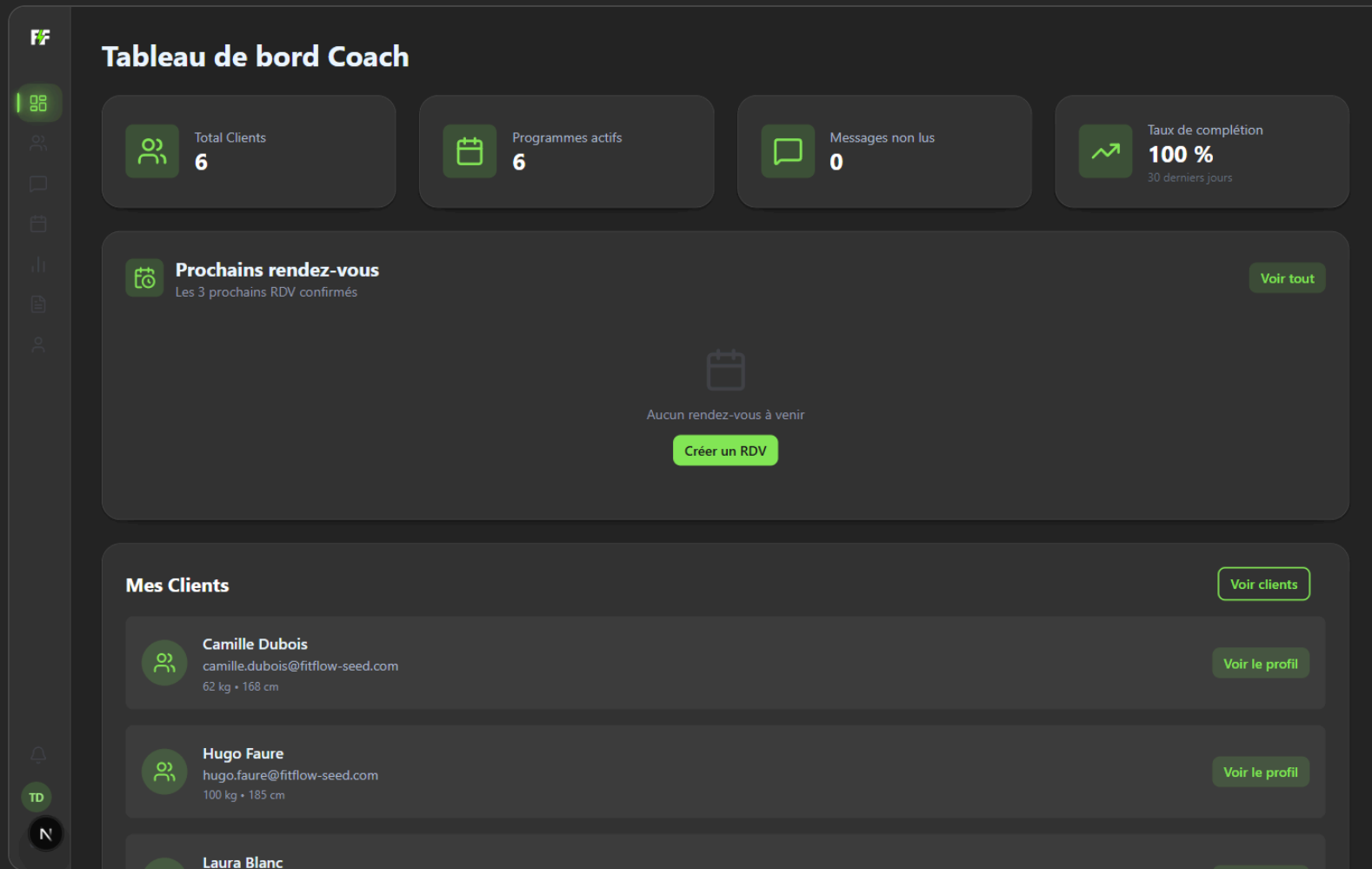
Séances créées par le client

Écartée volontairement, pour ne pas mélanger les rôles.

**FitFlow répond à un problème que j'ai observé,
pas à un problème que j'ai imaginé.**

Merci de votre attention.

Secours — le tableau de bord du coach

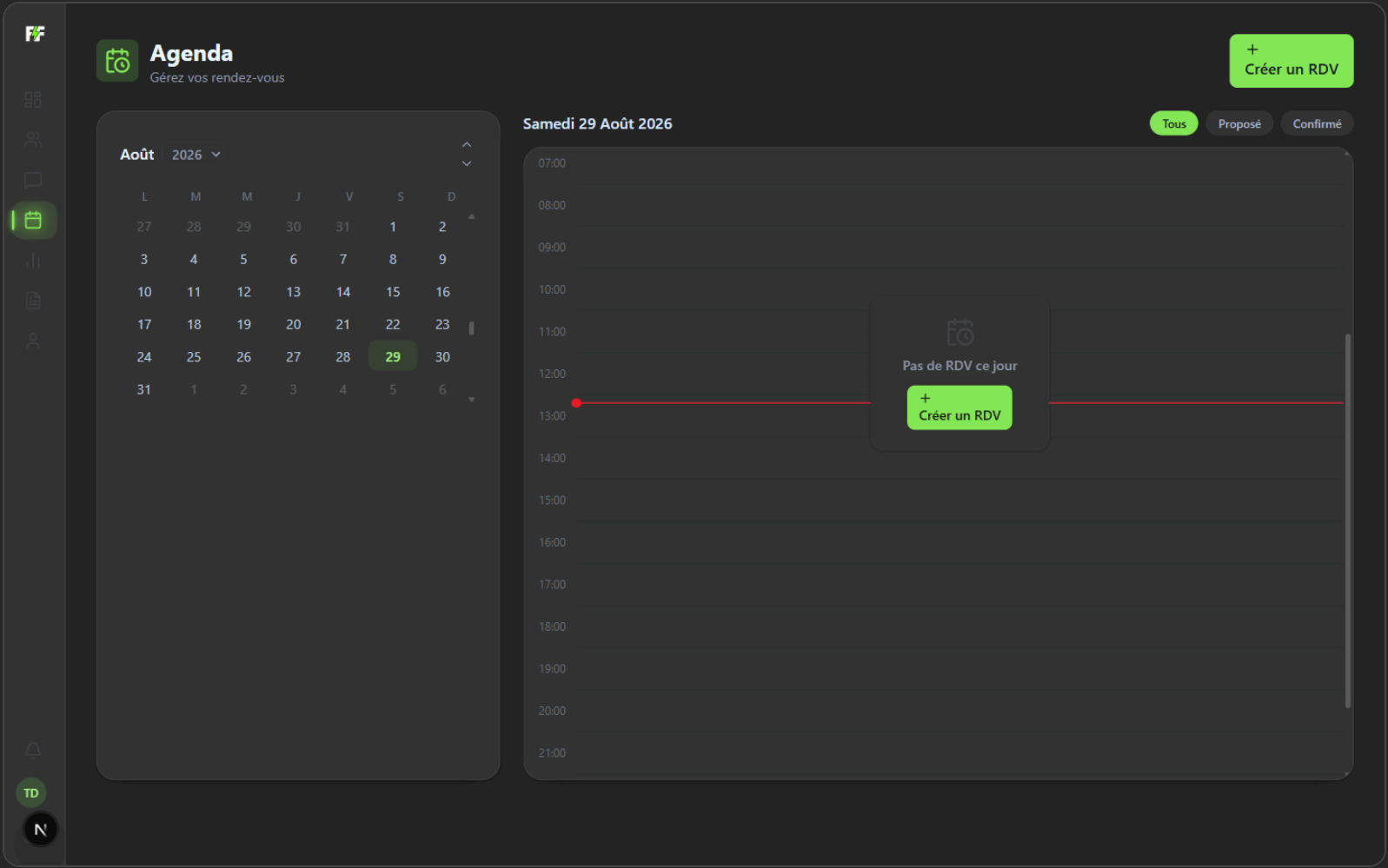


Étape 1 du parcours : le portefeuille de clients en un coup d'œil.

Secours — la fiche cliente

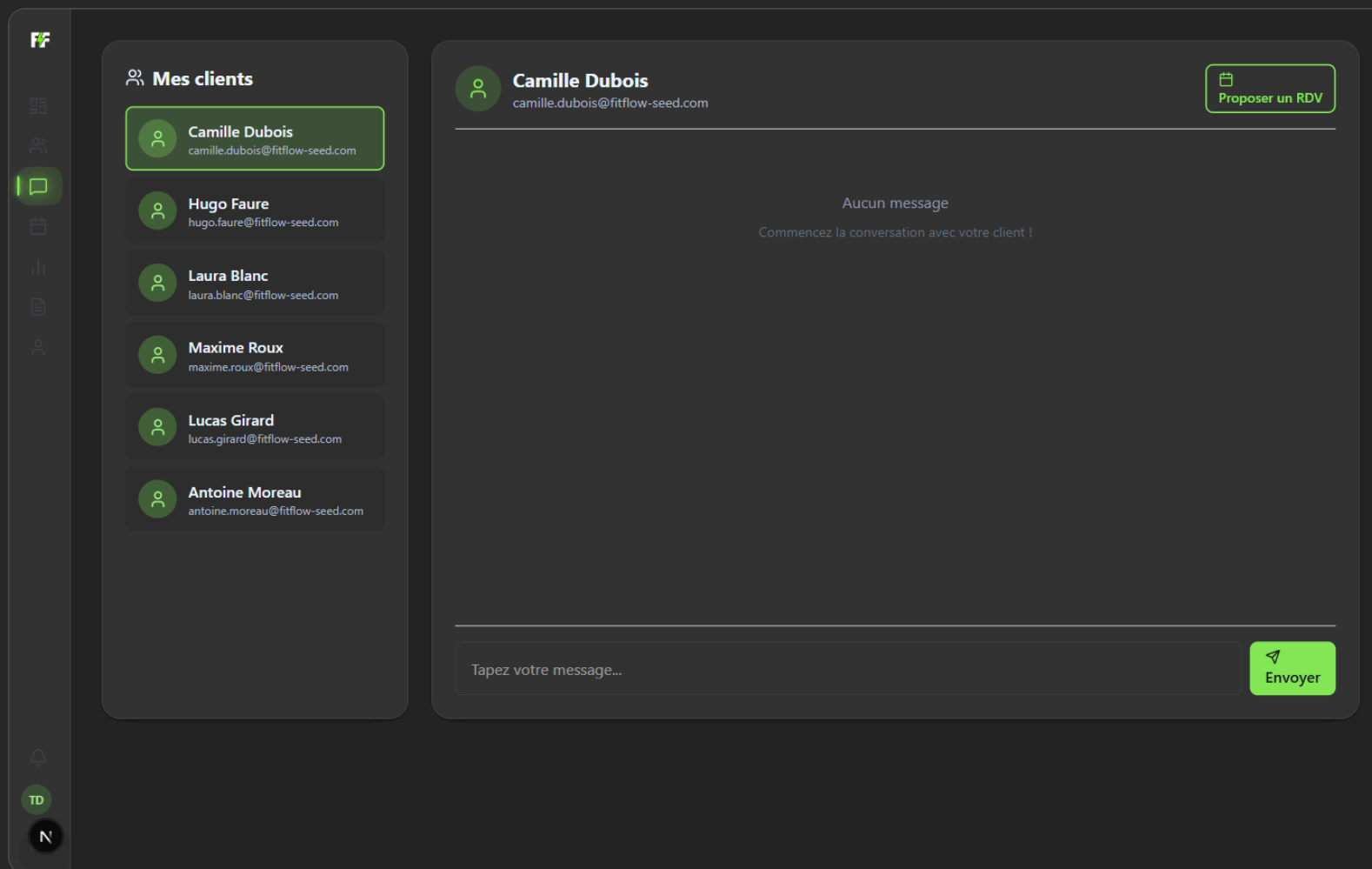
Étape 1 : programmes, statistiques et historique d'une cliente.

Secours — l'agenda



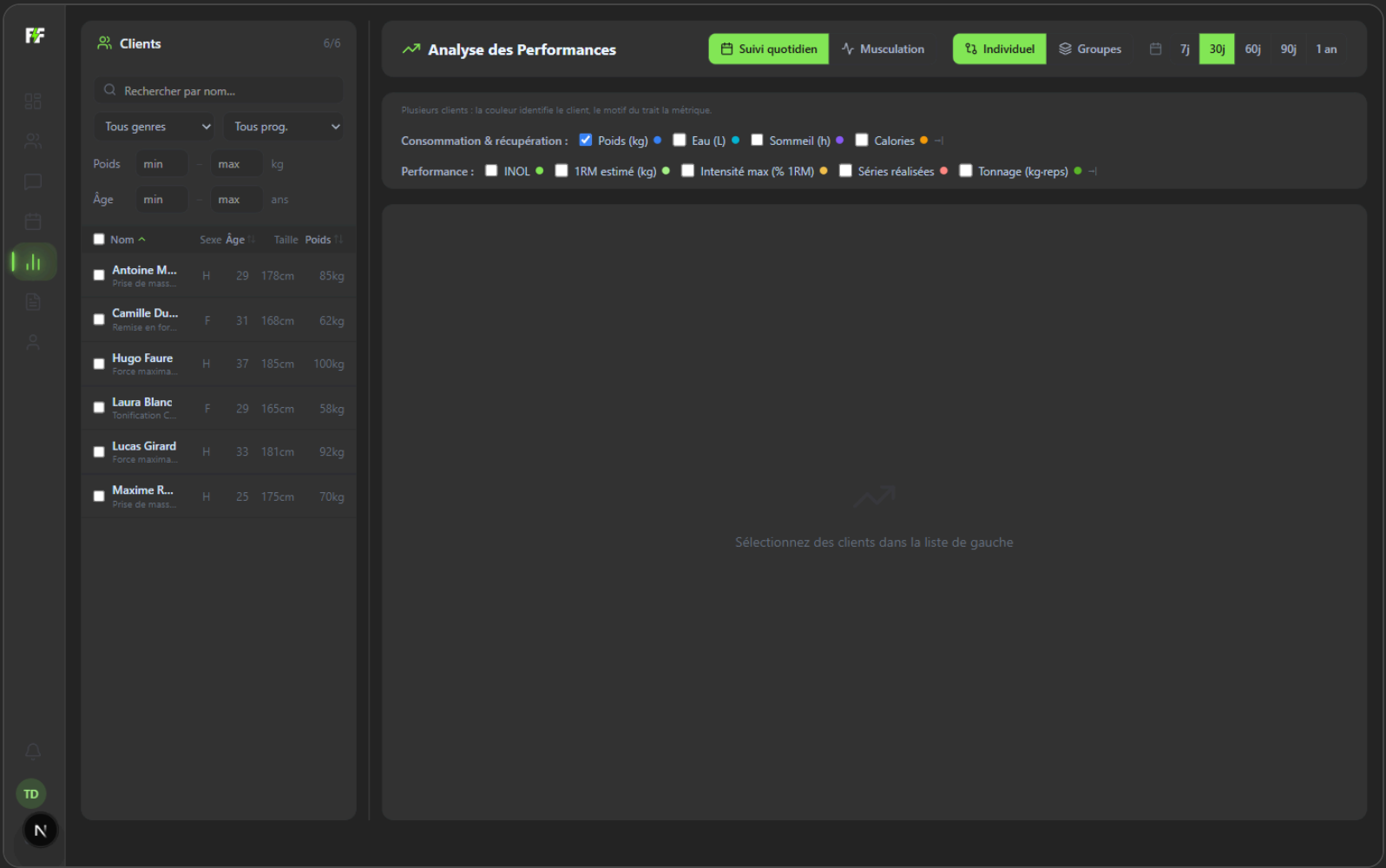
Étape 3 : le rendez-vous proposé depuis la conversation apparaît ici.

Secours — la messagerie



Étape 3 : la conversation, d'où part la proposition de rendez-vous.

Secours — l'analyse des performances



Étape 6 : la donnée saisie en salle, revenue chez le coach.